KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

정렬과 sorted, max, min

무엇을 기준으로 줄 세우고
가장 큰 값을 고를지 key로 정하기

```
sales = {"coffee": 12, "latte": 30,
         "cake": 8, "tea": 15}

sorted(sales, key=sales.get, reverse=True)
# ['latte', 'tea', 'coffee', 'cake']

max(sales, key=sales.get)
# 'latte'
```

오늘의 순서

1 왜 필요한가

max(sales)가 tea인 이유, key 그림, 가장 짧은 key

4–6쪽

2 sorted, sort, key

새 리스트와 원본, None 함정, reverse, 글자 비교, key=len, str.lower, int

7–12쪽

3 튜플과 딕셔너리 정렬

앞 칸부터 비교, m[1] 기준, 두 기준, 키와 값과 items, get

13–18쪽

4 max와 min

key로 고르기, 가장 많이 팔린 메뉴, default, 상위 3개

19–22쪽

5 오류와 함정

숫자와 글자 섞기, key에 괄호, lambda와 key= 빠뜨리기

23–25쪽

6 실습

판매 순위표, 가장 비싼 메뉴, 이름 길이순, 성적표

26–29쪽

3쪽은 노트북 준비, 30–31쪽은 막힐 때와 한 장 요약이에요.

들어가기

21에서 이어서 해요

- ✓ 리스트, 튜플, sort, sorted, max, min — 13
- ✓ 딕셔너리 get, items() — 15
- ✓ sorted(..., reverse=True) — 16, 20
- ✓ for, enumerate() — 13, 18
- ✓ lambda, map, filter — 21

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

기억이 안 나면 오른쪽에 적힌 챕터를 먼저 다시 봐요.

1. 왜 필요한가

가장 많이 팔린 메뉴를 물었는데 tea가 나와요

[1]
```
sales = {"coffee": 12, "latte": 30, "cake": 8, "tea": 15}
print(sorted(sales))
print(max(sales))
```
기대한 건 latte
Python

```
['cake', 'coffee', 'latte', 'tea']
tea
```

coffee	latte	cake	tea
12	30	8	15

판매량은 latte가 30잔으로 가장 많아요

키끼리 비교했어요

sorted와 max는 딕셔너리의 키만 봐요. 글자는 abc 순이라 tea가 가장 뒤, 곧 가장 커요.

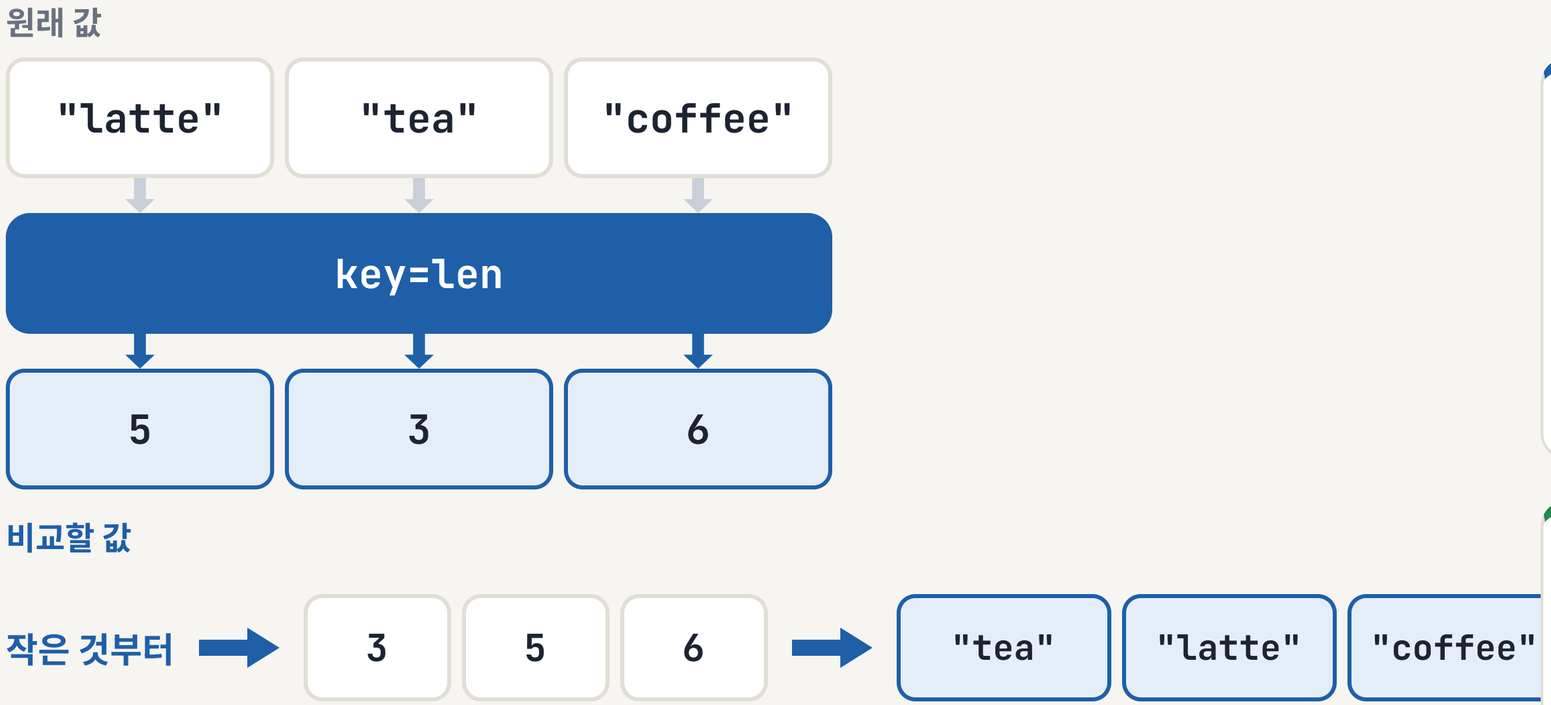
기준을 바꾸고 싶어요

이름 말고 판매량으로 비교하라고 알려 주는 방법이 key예요.

sorted와 max는 따로 말하지 않으면 값 자체를 비교해요.

1. 왜 필요한가

key는 비교할 값을 따로 만들어 줘요



key는 함수예요

값마다 key 함수를 써서 비교할 값을 만들어요. 21의 map처럼 하나씩 넣어요.

결과는 원래 값

줄은 비교할 값으로 세우고, 결과에는 원래 값이 그대로 들어가요.

key=함수를 넣으면 sorted, max, min이 그 함수의 결과로 비교해요.

1. 왜 필요한가

가장 짧은 key: 이름을 길이순으로

[2]
```python
names = ["latte", "tea", "coffee"]
print(sorted(names))
print(sorted(names, key=len))          # 길이로 비교
print(max(sales, key=sales.get))       # 판매량으로 비교
```

```
['coffee', 'latte', 'tea']
['tea', 'latte', 'coffee']
latte
```

key=len len 결과로 비교해요

key=sales.get 판매량으로 비교해요

괄호 없이 이름만 넣어요

21에서 map(len, names)처럼 함수를 넘긴 것과 같아요. sales.get도 키를 넣으면 값을 주는 함수예요. 3파트에서 다시 봐요.

이제 latte

앞 장에서 tea가 나왔던 max가 latte를 골라요.

key= 뒤에 함수를 넣으면 무엇으로 비교할지 바뀌어요.

2. sorted, sort, key

sorted는 새 리스트, sort는 원본을 바꿔요

[1]
```python
prices = [6500, 4500, 5000]
a = sorted(prices)
print(a)
print(prices)
prices.sort()
print(prices)
```
새 리스트 a

prices가 바뀌어요

Python

```
[4500, 5000, 6500]
[6500, 4500, 5000]
[4500, 5000, 6500]
```

`sorted(prices)` 함수예요. 줄 세운 새 리스트를 주고 prices는 그대로 둬요

`prices.sort()` 리스트의 메서드예요. prices 자체를 줄 세우고 돌려주는 값이 없어요

원본이 필요하면 sorted

가격표를 적은 순서도 나중에 필요하면 sorted를 써요. 줄 세운 결과만 있으면 되면 sort도 괜찮아요.

원본을 지키려면 sorted, 원본을 바꿔도 되면 sort를 써요.

2. sorted, sort, key

prices = prices.sort()를 하면 None만 남아요

```
[2] prices = [6500, 4500, 5000]
    prices = prices.sort()        돌려준 값은 None
    print(prices)                                  Python

    None
```

1	`prices.sort()`	prices를 줄 세워요
2	돌려준 값	None
3	`prices = None`	줄 세운 리스트를 잃어요

19의 return 없는 함수와 같아요

sort는 원본을 바꾸기만 하고 값을 돌려주지 않아요. 돌려받은 None을 prices에 넣었어요.

이렇게 고쳐요 둘 중 하나

```
prices.sort()

prices = sorted(prices)
```

sort는 그 줄 하나로 쓰고, 결과를 담으려면 sorted를 써요.

2. sorted, sort, key

큰 것부터 세우려면 reverse=True

[3]
```python
prices = [6500, 4500, 5000]
print(sorted(prices, reverse=True))
prices.sort(reverse=True)
print(prices)
```

```
[6500, 5000, 4500]
[6500, 5000, 4500]
```



20에서 본 키워드 인자

reverse의 기본값이 False라서 빼면 작은 것부터예요. True를 넣으면 거꾸로예요.

sort에도 똑같이

prices.sort(reverse=True)처럼 sort도 reverse와 key를 받아요.

reverse=True는 큰 것부터, 글자는 z부터 세워요.

2. sorted, sort, key

글자는 앞 글자부터 하나씩 비교해요

[4]
```python
print(sorted(["Latte", "cake", "Tea", "coffee"]))
print(sorted(["라떼", "커피", "녹차"]))
print(sorted(["10", "9", "100"]))
```

```
['Latte', 'Tea', 'cake', 'coffee']
['녹차', '라떼', '커피']
['10', '100', '9']
```

`"Tea" < "cake"`	`True`	대문자가 소문자보다 먼저예요. 대문자 A~Z 뒤에 소문자 a~z가 와요
`"녹차" < "라떼"`	`True`	한글은 가나다 순이에요
`"100" < "9"`	`True`	첫 글자 1과 9를 먼저 비교해요. 글자로 담긴 숫자는 크기순이 아니에요

글자는 사전처럼 첫 글자부터 비교해요. 숫자 크기로 세우려면 숫자로 바꿔요.

2. sorted, sort, key

key=len: 길이를 재서 비교해요

[5]
```python
names = ["latte", "tea", "coffee", "mocha"]
print(list(map(len, names)))
print(sorted(names, key=len))
print(sorted(names, key=len, reverse=True))
```

```
[5, 3, 6, 5]
['tea', 'latte', 'mocha', 'coffee']
['coffee', 'latte', 'mocha', 'tea']
```

names

"latte"	"tea"	"coffee"	"mocha"

len

5	3	6	5

21의 map(len, ...)과 같은 값

sorted가 안에서 이 길이 값으로 비교해요.

길이가 같으면 원래 순서

latte와 mocha는 둘 다 5예요. 처음 적은 순서대로 latte가 먼저 남아요.

key=len은 len 결과로, 같으면 원래 순서를 지켜 줄 세워요.

2. sorted, sort, key

key=str.lower, key=int로 비교 방법을 바꿔요

[6]
```python
print("Latte".lower())
print(sorted(["Latte", "cake", "Tea", "coffee"], key=str.lower))
print(sorted(["10", "9", "100"], key=int))
```

```
latte
['cake', 'coffee', 'Latte', 'Tea']
['9', '10', '100']
```

`"Latte".lower()` 글자를 모두 소문자로 바꾼 새 글자를 줘요

`key=str.lower` 소문자로 바꿔서 비교해요. 결과는 원래 글자

`key=int` 숫자로 바꿔서 크기순으로 비교해요

str.lower는 이름만

lower는 글자(str)의 메서드라서 str.lower로 적어 넘겨요.

결과는 그대로 두고 비교할 때만 바꾸고 싶으면 key에 바꾸는 함수를 넣어요.

3. 튜플과 딕셔너리 정렬

튜플은 앞 칸부터 비교해요

[1]
```python
items = [("latte", 5000), ("cake", 6500), ("coffee", 4500), ("cake", 6000)]
print(sorted(items))
```

```
[('cake', 6000), ('cake', 6500), ('coffee', 4500), ('latte', 5000)]
```

("cake" 6000) < ("cake" 6500)

첫 칸이 같아서 둘째 칸 6000과 6500을 비교해요

("coffee" 4500) < ("latte" 5000)

첫 칸이 다르면 둘째 칸은 보지 않아요

글자 비교와 같은 방식

글자가 첫 글자부터 비교했듯이 튜플은 첫 칸부터 비교해요. 그래서 그냥 sorted하면 이름순이에요.

튜플 리스트를 그냥 줄 세우면 첫 칸 기준, 같으면 둘째 칸 기준이에요.

3. 튜플과 딕셔너리 정렬

key=lambda m: m[1]로 둘째 칸 기준

[2]
```python
items = [("latte", 5000), ("cake", 6500), ("coffee", 4500)]
print(sorted(items, key=lambda m: m[1]))
print(sorted(items, key=lambda m: m[1], reverse=True))
```

```
[('coffee', 4500), ('latte', 5000), ('cake', 6500)]
[('cake', 6500), ('latte', 5000), ('coffee', 4500)]
```



m은 튜플 하나

lambda의 m에 튜플이 하나씩 들어가요. m[1]이 가격이라 가격으로 비교해요.

원하는 칸으로 세우려면 그 칸을 꺼내는 lambda를 key에 넣어요.

3. 튜플과 딕셔너리 정렬

판매량 많은 순, 같으면 이름순으로 세워요

[3]
```python
sales = [("latte", 30), ("cake", 8), ("coffee", 30), ("tea", 15)]
print(sorted(sales, key=lambda m: m[1], reverse=True))      30끼리는 원래 순서
print(list(map(lambda m: (-m[1], m[0]), sales)))
print(sorted(sales, key=lambda m: (-m[1], m[0])))           같으면 이름순
```
Python

```
[('latte', 30), ('coffee', 30), ('tea', 15), ('cake', 8)]
[(-30, 'latte'), (-8, 'cake'), (-30, 'coffee'), (-15, 'tea')]
[('coffee', 30), ('latte', 30), ('tea', 15), ('cake', 8)]
```

튜플	key 결과	읽는 법
("latte", 30)	(-30, 'latte')	-30이 가장 작아서 앞으로
("coffee", 30)	(-30, 'coffee')	-30이 같아서 이름으로, coffee가 먼저
("tea", 15)	(-15, 'tea')	다음은 -15
("cake", 8)	(-8, 'cake')	-8이 가장 커서 맨 뒤

빼기로 뒤집기

숫자 앞에 -를 붙이면 큰 수가 작은 수가 돼요. 판매량은 많은 순, 이름은 abc 순으로 한 번에 세울 수 있어요.

3. 튜플과 딕셔너리 정렬

딕셔너리는 키, 값, 짝 중 무엇을 줄 세울지 골라요

[4]
```python
sales = {"coffee": 12, "latte": 30, "cake": 8, "tea": 15}
print(sorted(sales))
print(sorted(sales.values()))
print(sorted(sales.items()))
```

Python

```
['cake', 'coffee', 'latte', 'tea']
[8, 12, 15, 30]
[('cake', 8), ('coffee', 12), ('latte', 30), ('tea', 15)]
```

`sorted(sales)` **키만** 이름이 abc 순으로. 판매량은 빠져요

`sorted(sales.values())` **값만** 판매량이 작은 순으로. 어느 메뉴인지 빠져요

`sorted(sales.items())` **(키, 값)** 튜플 리스트가 돼요. 첫 칸인 이름 순이에요

items()에 key=lambda kv: kv[1]로 판매 순위

[5]
```python
rank = sorted(sales.items(), key=lambda kv: kv[1], reverse=True)
print(rank)
for name, n in rank:
    print(f"{name:<7}{n:>3}잔")
```

```
[('latte', 30), ('tea', 15), ('coffee', 12), ('cake', 8)]
latte   30잔
tea     15잔
coffee  12잔
cake     8잔
```

("latte" 30) kv[0] 이름 kv[1] 판매량

kv는 짝 하나

key와 value를 줄여 kv라고 지었어요. 이름은 자유예요.

13의 꺼내 담기

for name, n in rank:로 튜플을 두 변수에 나눠 담아 찍어요.

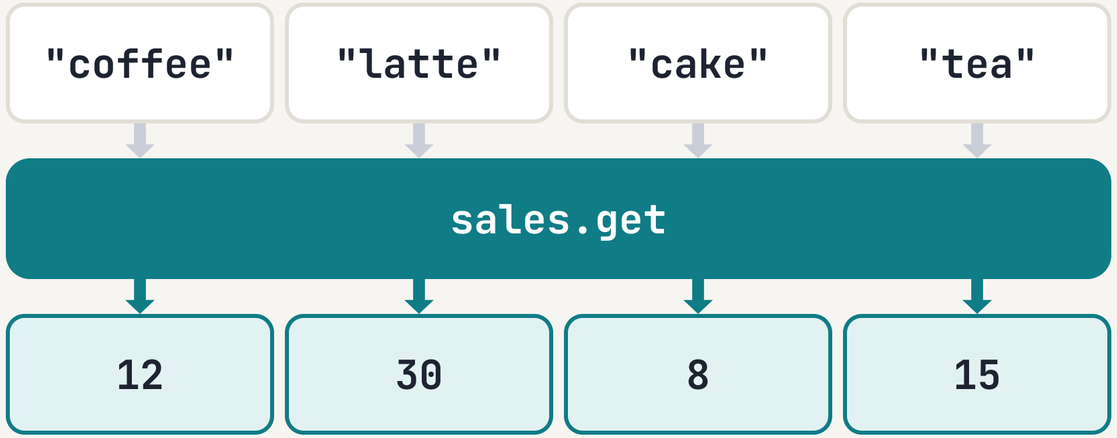
이름과 값을 같이 두고 값으로 세우려면 items()에 key를 줘요.

3. 튜플과 딕셔너리 정렬

키만 필요하면 key=sales.get

[6]
```python
print(sales.get("latte"))
print(sorted(sales, key=sales.get))
print(sorted(sales, key=sales.get, reverse=True))
```
Python

```
30
['cake', 'coffee', 'tea', 'latte']
['latte', 'tea', 'coffee', 'cake']
```



get은 키를 넣으면 값을 줘요

15의 sales.get("latte")예요. 괄호 없이 sales.get만 넘기면 함수 자체가 넘어가요.

lambda로 써도 같아요

key=lambda k: sales[k]도 키마다 판매량을 꺼내요.

키를 값 순서로 세우려면 key에 sales.get을 넘겨요.

4. max와 min

max와 min도 key를 받아요

[1]
```python
prices = [4500, 5000, 6500, 3000]
print(max(prices), min(prices))
names = ["latte", "tea", "coffee"]
print(max(names), min(names))
print(max(names, key=len))
```
가장 긴 이름

Python

```
6500 3000
tea coffee
coffee
```

sorted 맨 앞, 맨 뒤와 같아요

min은 sorted 결과의 첫 값, max는 마지막 값과 같아요. 하나만 필요하면 max, min이 짧아요.

sorted와 같은 비교

숫자는 크기, 글자는 abc 순이에요. tea가 가장 크고 coffee가 가장 작아요.

결과는 원래 값 하나

key=len으로 길이를 비교해도 돌아오는 건 길이 6이 아니라 이름 coffee예요.

max(xs, key=함수)는 함수 결과가 가장 큰 원래 값을 돌려줘요.

4. max와 min

가장 많이 팔린 메뉴는 max(sales, key=sales.get)

[2]
```python
sales = {"coffee": 12, "latte": 30, "cake": 8, "tea": 15}
print(max(sales))
print(max(sales, key=sales.get))
print(min(sales, key=sales.get))
print(max(sales.items(), key=lambda kv: kv[1]))
```
1파트의 tea

Python

```
tea
latte
cake
('latte', 30)
```

coffee	latte	cake	tea
12	30	8	15

key=sales.get 이름만 받아요

min(…) 가장 적게 팔린 cake

items(), kv[1] 이름과 판매량을 같이

필요한 것만 골라 써요

이름만이면 get, 둘 다면 items()예요.

4. max와 min

빈 리스트에 max를 쓰면 ValueError가 나요

[3]
```
print(max([]))
```
Python

```
ValueError: max() iterable argument is empty
```

[4]
```
big = list(filter(lambda p: p > 10000, prices))
print(big)
print(max(big, default=0))
```
비면 0 Python

```
[]
0
```

filter 결과는 비어 있을 수 있어요

1만 원이 넘는 메뉴가 없어서 big이 빈 리스트예요. 고른 결과에 max를 쓸 땐 default를 같이 적어요.

가장 큰 값이 없어요

값이 하나도 없으면 고를 게 없어서 오류가 나요. min도 같아요.

default=값

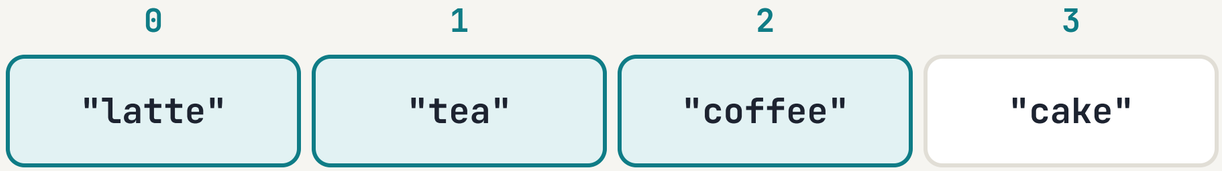
비어 있을 때 대신 돌려줄 값이에요. 15의 get(키, 기본값)과 비슷해요.

4. max와 min

sum, len, 자르기와 같이 써요

[5]
```python
print(sum(sales.values()), len(sales))
print(sum(sales.values()) / len(sales))
top3 = sorted(sales, key=sales.get, reverse=True)[:3]
print(top3)
```
Python

```
65 4
16.25
['latte', 'tea', 'coffee']
```



sorted가 리스트를 주니까 13의 자르기를 바로 붙여요

`sum, len` 모두 65잔, 메뉴 4개

`sum / len` 메뉴 하나당 평균 16.25잔

`[:3]` 줄 세운 리스트에서 앞 3개

상위 몇 개는 sorted 뒤에 [:n]을 붙여서 뽑아요.

5. 오류와 함정

숫자와 글자를 섞으면 비교할 수 없어요

[1]
```python
print(sorted([4500, "5000", 6500]))
```

```
TypeError: '<' not supported between instances of 'str' and 'int'
```

[2]
```python
print(sorted([4500, "5000", 6500], key=int))
```

```
[4500, '5000', 6500]
```

'<' not supported

글자 "5000"과 숫자 4500 중 무엇이 작은지 정할 수 없다는 뜻이에요. max, min도 같은 오류가 나요.

key=int로 비교만 숫자로

결과에는 "5000"이 글자로 남아 있어요. 처음부터 같은 타입으로 담는 게 가장 좋아요.

한 리스트에는 같은 타입을 담아요. 섞였으면 key로 맞춰 비교해요.

5. 오류와 함정

key에 괄호를 붙이면 TypeError가 나요

[3]
```python
print(sorted(["latte", "tea"], key=len()))
```

```
TypeError: len() takes exactly one argument (0 given)
```

[4]
```python
sales = {"coffee": 12, "latte": 30, "cake": 8, "tea": 15}
print(sorted(sales, key=sales.get()))
```

```
TypeError: get expected at least 1 argument, got 0
```

괄호를 붙이면 그 자리에서 아무것도 넣지 않고 실행해요

len()과 sales.get()이 값 없이 먼저 불려서 0 given, got 0이 나왔어요. 21의 map(add10())과 같은 실수예요.

`key=len()` → `key=len`

`key=sales.get()` → `key=sales.get`

5. 오류와 함정

lambda나 key=를 빠뜨리면 생기는 오류

[5]
```python
print(sorted(sales.items(), key=kv[1]))
```

```
NameError: name 'kv' is not defined
```

[6]
```python
print(sorted(sales, sales.get))
```

```
TypeError: sorted expected 1 argument, got 2
```

이렇게 고쳐요

```
sorted(sales.items(), key=lambda kv: kv[1])
sorted(sales, key=sales.get)
```

lambda kv: 가 빠졌어요

kv는 lambda가 만드는 칸 이름이에요. lambda 없이 쓰면 그런 변수가 없어요.

key= 이름을 빠뜨렸어요

sorted는 줄 세울 값 하나만 위치로 받아요. key는 꼭 key=로 적어요.

6. 실습

판매 순위표 3위까지 찍기

[1]
```python
sales = {"coffee": 12, "latte": 30, "cake": 8, "tea": 15, "juice": 15}
rank = sorted(sales.items(), key=lambda kv: (-kv[1], kv[0]))
for i, kv in enumerate(rank[:3], 1):
    print(f"{i}위 {kv[0]:<6}{kv[1]:>3}잔")
```

Python

```
1위 latte  30잔
2위 juice  15잔
3위 tea    15잔
```

kv는 튜플

kv[0]이 이름, kv[1]이 잔 수예요.

`(-kv[1], kv[0])` 3파트의 두 기준. juice와 tea가 15로 같아요

`enumerate(…, 1)` 18에서 배운 번호 붙이기. 1위부터

줄 세우기, 자르기, 번호 붙이기를 이어서 순위표를 만들어요.

6. 실습

가장 비싼 메뉴, 가장 싼 메뉴, 가격순 메뉴판

[2]

```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500, "tea": 4000}
hi = max(menu, key=menu.get)
lo = min(menu, key=menu.get)
print(f"가장 비싼 메뉴 {hi} {menu[hi]:,}원")
print(f"가장 싼 메뉴 {lo} {menu[lo]:,}원")
for m in sorted(menu, key=menu.get):
    print(f"{m:<7}{menu[m]:>6,}원")
```

이름을 받아요

가격순 키

Python

```
가장 비싼 메뉴 cake 6,500원
가장 싼 메뉴 tea 4,000원
tea     4,000원
coffee  4,500원
latte   5,000원
cake    6,500원
```

이름으로 값 꺼내기

max가 이름을 주니까 menu[hi]로 가격을 꺼내 같이 찍어요.

for와 바로 써요

sorted가 준 키 리스트를 for로 돌아요.

6. 실습

메뉴판을 이름 길이순, 같으면 abc 순으로

[3]
```python
names = ["mocha", "tea", "latte", "americano", "cake"]
print(sorted(names, key=len))                          # 5끼리는 원래 순서
print(sorted(names, key=lambda n: (len(n), n)))        # 같으면 abc
```

```
['tea', 'cake', 'mocha', 'latte', 'americano']
['tea', 'cake', 'latte', 'mocha', 'americano']
```

key 결과 (5 "mocha") > (5 "latte")

첫 칸 5가 같아서 둘째 칸 이름으로 비교해요. latte가 앞이에요

같을 때 기준 하나 더

튜플로 돌려주면 첫 기준이 같을 때 둘째 기준을 봐요.

기준이 둘이면 key가 (첫째 기준, 둘째 기준) 튜플을 돌려주게 해요.

6. 실습

성적표를 점수순으로 세우고 1등과 평균 구하기

[4]

```python
scores = [("kim", 92), ("lee", 78), ("park", 85), ("choi", 59)]
ranked = sorted(scores, key=lambda s: s[1], reverse=True)      # 점수 높은 순
for name, score in ranked:
    print(f"{name:<5}{score:>3}")
print("1등:", max(scores, key=lambda s: s[1])[0])              # 튜플에서 이름만
print("평균:", sum(map(lambda s: s[1], scores)) / len(scores))  # 21의 map
```

Python

```
kim   92
park  85
lee   78
choi  59
1등: kim
평균: 78.5
```

[0]을 붙여 이름만

max가 튜플 ('kim', 92)를 주니까 [0]으로 이름을 꺼내요.

점수만 모아 더하기

map으로 점수만 꺼내 sum에 넘겨요.

정리

막히면 오류의 마지막 줄을 봐요

1	**max(sales)가 엉뚱한 이름이에요** 딕셔너리는 키끼리 abc로 비교해요	`key=sales.get`
2	**prices를 찍으니 None이에요** prices = prices.sort()로 받았어요	**prices.sort()만**
3	**['10', '100', '9'] 순서예요** 글자는 앞 글자부터 비교해요	`key=int`
4	**`TypeError: '<' not supported between ...`** 숫자와 글자를 섞었어요	**타입 맞추기, key=int**
5	**`TypeError: len() takes exactly one argument`** key에 괄호를 붙였어요	`key=len`
6	**`NameError: name 'kv' is not defined`** lambda kv: 를 빠뜨렸어요	`key=lambda kv: kv[1]`
7	**`ValueError: max() iterable argument is empty`** 빈 리스트에 max를 썼어요	`default=0`

순서가 이상하면 map(key 함수, 리스트)로 비교할 값을 먼저 찍어 봐요.

한 장 요약

`sorted(xs)`

새 리스트, 원본 그대로

`xs.sort()`

원본을 바꾸고 None

`key=len`

len 결과로 비교

`key=lambda m: m[1]`

튜플 둘째 칸으로

`key=lambda m: (-m[1], m[0])`

많은 순, 같으면 이름순

`sorted(d, key=d.get)`

값 순서로 키 세우기

`max(d, key=d.get)`

값이 가장 큰 키

`max(xs, default=0)`

비어 있으면 0

key에는 괄호 없이 함수 이름이나 lambda를 넣어요

다음 챕터: 리스트 컴프리헨션으로 고르고 바꾸기를 한 줄에